

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 3 – Mock Interview

Course Code:	ITA05	Course Title:	Computer Vision
CO Assessed:	CO5 – Naturalization (independent application using OpenCV)P5	Assessment:	Assessment Tool 3 – Mock Interview
Weightage:	25% of CIA	Total Marks:	100
CO4 Statement:	Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL6)		
Student Name:	YAVANIKA S	Reg. No.:	192521258
Section / Batch:	ITA0512	Date:	24.08.2026

Code of Conduct

I Yavanika S [192521258] (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Yavanika

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning	Marks
Technical Knowledge	25	Demonstrates strong and accurate subject knowledge with in-depth understanding	Shows good knowledge with minor gaps	Basic knowledge with noticeable errors	Lacks fundamental technical knowledge	
Problem Solving	25	Solves problems logically and applies concepts effectively in real scenarios	Applies concepts with moderate accuracy	Limited problem-solving ability; requires guidance	Unable to solve or apply concepts	
Analytical Thinking	15	Analyzes questions critically and provides well-structured, logical answers	Provides reasonable analysis with some structure	Superficial or partially structured responses	No analytical thinking demonstrated	
Communication	15	Communicates clearly, confidently, and professionally with proper terminology	Communicates well with minor hesitation	Communication lacks clarity or confidence	Unable to communicate effectively	
Confidence	10	Displays high confidence, positive attitude, and professional behavior throughout	Shows good confidence with minor issues	Low confidence; inconsistent behavior	Lacks confidence and professionalism	
Response to Questions	10	Responds accurately, promptly, and elaborates with relevant examples	Responds correctly but with limited depth	Partial responses; needs prompting	Unable to respond appropriately	

Question 11

"compare two images and highlight differences visually.

Analyse the problem and design a solution, justifying your approach."

1. Clarifying the Problem

Before designing anything, I'd confirm the scope out loud with the interviewer:

- What counts as a "difference"? — Pixel-level changes (an object added/removed/moved), not just brightness or JPEG-compression noise.
- Are the two images already aligned (same camera angle, same size)? — I'll assume near-aligned but not guaranteed identical size, so I handle resizing.
- What's the output? — A visual result: both images with the changed regions boxed, plus an optional difference mask, so a human can see exactly what changed.
- Real-time or one-off comparison? — This is a one-off / batch comparison (e.g., before-after photos), not a live video stream, so heavy real-time optimization isn't the priority — accuracy is.

2. My Approach

I'd solve this in three stages: (1) make both images directly comparable, (2) compute a difference map that is robust to minor noise rather than a raw pixel subtraction, and (3) turn that difference map into a clean visual — bounding boxes around only the regions that genuinely changed.

The key design decision is using Structural Similarity (SSIM) instead of a plain `cv2.absdiff()`. A raw pixel difference flags every tiny lighting or compression variation as "different," which floods the result with false positives. SSIM compares local structure (luminance, contrast, texture) in small windows, so it stays quiet on noise and lights up only where the actual content changed — exactly what "highlight differences visually" needs.

3. Solution Pipeline

SOLUTION PIPELINE: Visual Difference Highlighting Between Two Images

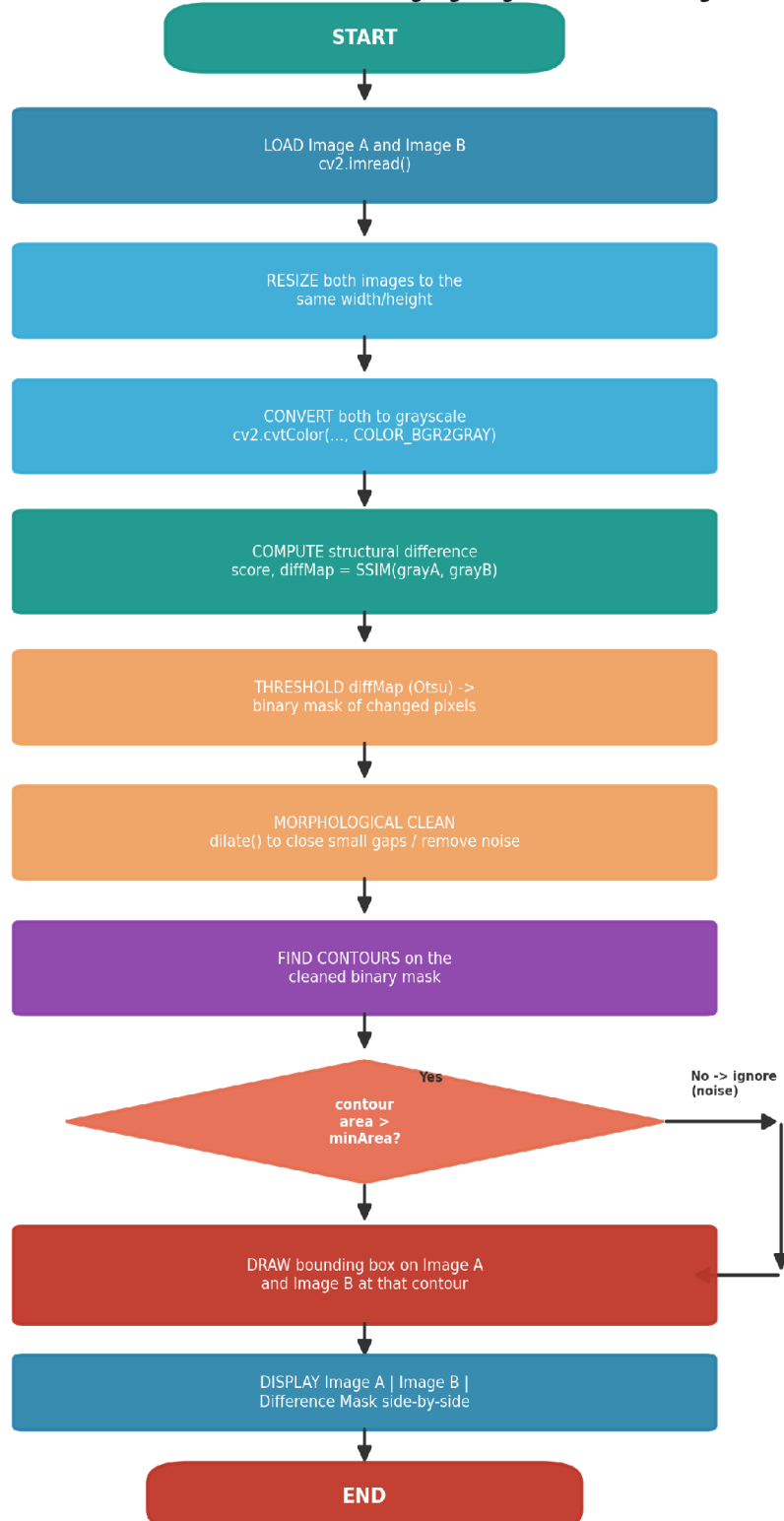


Figure 1: End-to-end pipeline for visual difference highlighting

4. Implementation (OpenCV)

This is the function I'd actually write and talk through on a whiteboard/IDE:

```
highlight_differences.py
```

```
import cv2
from skimage.metrics import structural_similarity as ssim
def highlight_differences(pathA, pathB, min_area=150):
    imgA = cv2.imread(pathA)
    imgB = cv2.imread(pathB)
    imgB = cv2.resize(imgB, (imgA.shape[1], imgA.shape[0])) # match sizes
    grayA = cv2.cvtColor(imgA, cv2.COLOR_BGR2GRAY)
    grayB = cv2.cvtColor(imgB, cv2.COLOR_BGR2GRAY)
    score, diff = ssim(grayA, grayB, full=True) # structural, not pixel diff
    diff = (diff * 255).astype("uint8")
    thresh = cv2.threshold(diff, 0, 255,
                           cv2.THRESH_BINARY_INV | cv2.THRESH_OTSU)[1]
    thresh = cv2.dilate(thresh, None, iterations=2) # close small gaps
    contours, _ = cv2.findContours(thresh, cv2.RETR_EXTERNAL,
                                   cv2.CHAIN_APPROX_SIMPLE)
    for c in contours:
        if cv2.contourArea(c) < min_area: # drop noise blobs
            continue
        x, y, w, h = cv2.boundingRect(c)
        cv2.rectangle(imgA, (x, y), (x + w, y + h), (0, 0, 255), 2)
        cv2.rectangle(imgB, (x, y), (x + w, y + h), (0, 0, 255), 2)
    print(f'Similarity score: {score:.4f}')
    return imgA, imgB, thresh
```

5. What the Output Looks Like

Conceptual Output: Detecting & Highlighting Differences Between Two Images

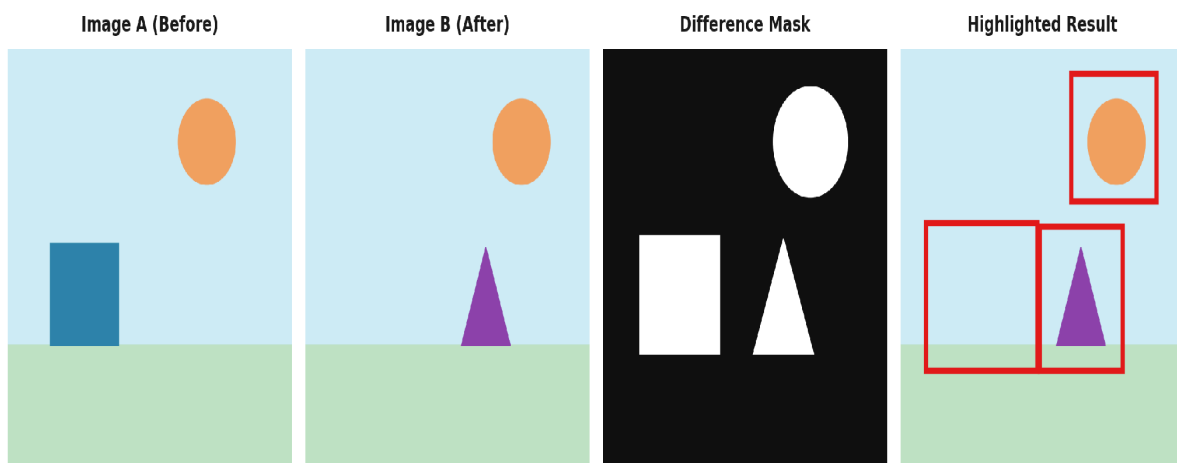


Figure 2: Image A vs Image B, the resulting difference mask, and the final boxed output

6. Justifying the Approach

- SSIM over absdiff — SSIM is robust to lighting/compression noise and mirrors how a human perceives "changed," so the boxes land on real content changes, not sensor noise.
- Otsu's threshold instead of a fixed number — the right cut-off between "changed" and "unchanged" pixels varies per image pair; Otsu picks it automatically from the diff map's histogram.
- Dilation before contour detection — a changed object often produces several nearby broken blobs; dilating merges them into one clean region instead of many fragmented boxes.
- min_area filter — removes tiny leftover noise contours so the interviewer sees only meaningful differences, not visual clutter.
- Bounding boxes on both images — lets a viewer flip between "before" and "after" and see exactly the same regions marked, which is the actual goal: highlight, not just detect.

7. Edge Cases I'd Call Out

- Different image sizes — resized in step 1 before any pixel-level comparison happens.
- Slight camera shift / misalignment — for a stronger solution I'd mention feature-based alignment (ORB keypoints + homography via `cv2.findHomography`) before the diff step, if the two shots aren't guaranteed to be pixel-aligned.
- Large lighting change across the whole frame — could be normalized first with histogram equalization so SSIM isn't dominated by a global brightness shift.
- Many small scattered changes (e.g., textured background) — min_area and dilation parameters would need tuning per use case; I'd expose them as function arguments, as done above.

8. Interview Answer

"I'd align and grayscale both images, compute an SSIM-based structural difference map instead of a raw pixel diff to stay robust to noise, threshold it with Otsu, clean it up with dilation, then find contours above a minimum area and draw bounding boxes on both images — giving a clean visual of exactly what changed."